

EECS 3311 Stage 1 Design Report

Project Name : PulseForgeAI - Biometric-Driven Gym & Recovery Agent

Repository :

Project Overview & Motivation

1.1 Project Description

1. What problem does your project solve?

PulseForge AI completes information provided by smart watches. Let's take for example a person named Joe. Now Joe is beginning to take the gym, his sleep and diet seriously. Now he is doing great, tracking all his calories, working out consistently and sleeping well. But he does all these things separately. He tracks his gym session, then his sleep and then how he eats. He is following a consistent plan that could either harm him or not allow him to become better. PulseForge AI will solve the disconnect between each microservice by using real data to dynamically adapt accurate training volume, exercise intensity and physiology readiness.

2. Who are the target users?

- **University & College Students:**

Students who balance intensive academic workloads, irregular study schedules, and shifting sleep patterns alongside resistance training. While they may follow a consistent workout routine, academic stress, cognitive strain, and acute sleep debt frequently impair physical recovery. PulseForge AI assists students by detecting hidden physiological deficits (via suppressed HRV and elevated resting heart rate) and automatically adjusting workout volume, preventing burnout and injury during exam periods.

- **Competitive & Recreational Athletes:**

Trainees who adhere to disciplined, progressive overload regimes and require precise physiological calibration to maximize muscular adaptations and athletic performance. Rather than following static programming, these users utilize the agent to monitor nervous system fatigue and auto-regulate training loads, ensuring peak readiness for heavy lifts while preventing overtraining plateaus.

- **Working Professionals (Desk & Manual Labor):**

Individuals navigating occupational stressors that accumulate systemic fatigue. Desk workers frequently deal with postural fatigue, sedentary work hours, and high mental stress, whereas manual laborers incur high daily physical energy expenditure. The agent bridges this gap by calculating true active energy burn against logged caloric intake and dynamically recommending suitable exercise variations and recovery protocols.

- **General Fitness Enthusiasts ("Everyday Wearable Users"):**

Everyday individuals who track biometrics using an Apple Watch or fitness tracker but lack the sports science expertise to translate raw metrics (such as sleep phases or HRV readings) into actionable gym decisions. PulseForge AI

acts as an accessible digital coach, interpreting complex health metrics into simple, structured workout plans and nutritional feedback.

3. What can the agent do?

PulseForge AI acts as an autonomous workout and recovery coach:

- **Ingests & Normalizes Biometrics:** Parses raw wearable exports (XML/JSON/CSV) to extract sleep duration, resting HR, HRV, and active energy.
- **Calibrates Workouts in Real Time:** Reasonably alters exercise selection, sets, rep targets, and Rate of Perceived Exertion (RPE) to match daily recovery capacity.
- **Detects Systemic Fatigue & Overtraining:** Identifies negative trends across consecutive days (e.g., suppressed HRV combined with high lifting tonnage) and schedules automatic deload periods.
- **Manages Live Gym Sessions:** Tracks active sets and weights lifted with an undoable command history and dynamic rest timers.
- **Context-Aware Training Q&A:** Answers user queries regarding training modifications, exercise substitutions, and form adjustments grounded directly in their personal health metrics.
- **A Personal Jarvis:** Monitoring live data to ensure positive haptic feedback for physical wellbeing including proper heart activity and sleep. Triggers alerts for fluctuations

4. Why is an AI agent appropriate for this problem?

On all smart watches, we are just provided with the biometric data. More can be done with this data to ensure proper planning, positive development, and a support that most beginners need as improving well being can be confusing. The agent will be of continuous support.

5. Which AI/LLM model(s) do you plan to use?

PulseForge AI integrates **Google Gemini 1.5 Flash**. Why?

- **Low Latency & High Throughput:** Enables near-instant workout generation and responsive interactive chat within both the GUI and CLI.
- **Strict JSON Schema Enforcement:** Gemini 1.5 Flash supports structured JSON output modes, guaranteeing that generated workout routines, sets, and rep prescriptions deserialize directly into internal Java/Python domain objects without syntax failures.
- **Large Context Window:** Readily ingests multi-week biometric trends, exercise history tables, and physiological notes without truncating relevant historical data.
- **Architectural Decoupling:** The LLM is wrapped behind an internal LLMClient interface, allowing the system to seamlessly switch to models like OpenAI's

gpt-4o-mini or an offline local model (e.g., llama3.2:3b via Ollama) without modifying business logic

6. How will the AI model interact with the rest of the software system?

The AI model does not interact directly with user interfaces or raw files. It is coordinated strictly through an internal **Agent Controller and Facade Layer**:

- **Trigger & Request Gathering:** The user triggers an action (via GUI or CLI). The controller gathers relevant domain state: calculated readiness score, recent workout logs, and target muscle split.
- **Context Assembling & Tool Invocation:** The agent framework constructs a structured prompt injecting deterministic baselines, historical exercise performance, and constraint boundaries..

1.2 Feature Specification

F01 – Multi-Source Biometric & Health Ingestion

- **Description:** Ingests health files (Apple Health export.xml, Garmin JSON, or manual CSV) and parses multi-domain records into structured biometric entities (sleep duration, deep/REM sleep, resting heart rate, HRV, and active energy burned).
- **User Interaction:** The user navigates to the "Biometrics" screen on the GUI and clicks "Import Health File", opening a file chooser dialog to select the file.
- **Input:** File path to an Apple Health XML or JSON export file.
- **Output:** A collection of normalized BiometricRecord domain objects stored in the user profile repository.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The UI forwards the path to the controller, which invokes AppleHealthXmlAdapter. The adapter parses specific sensor tags, creates BiometricRecord instances, validates non-null timestamp ranges, and persists them into ProfileRepository.
- **Error/Alternative Cases:** If the file format is corrupt or missing mandatory fields, the adapter raises an InvalidHealthDataException, which the UI catches to display an error dialog prompting for a valid export.

F02 – Multimodal Meal Photo Calorie & Macronutrient Analyzer

- Description: Analyzes an uploaded photograph of a meal using a vision-capable LLM to recognize food items, estimate portion sizes, and calculate nutritional values (calories, protein, carbohydrates, fats).
- User Interaction: On the "Diet & Nutrition" tab, the user clicks "Upload Meal Image", selects a photo (e.g., PNG/JPEG), views a preview thumbnail, and clicks "Analyze Meal".
- Input: Image file (binary path) and optional meal notes (e.g., "lunch, post-workout").
- Output: A MealLog object containing detected foods, estimated calories, macronutrient grams, and an editable confirmation table.
- AI Involvement: AI-Based.
- Expected Workflow: The image is base64-encoded and packaged into a multimodal prompt by VisionNutritionService. The LLM client returns structured JSON with itemized foods and macronutrients. The system creates a MealLog instance, and the GUI displays an editable preview for the user to confirm or tweak.
- Error/Alternative Cases: If the image is blurry, unrecognizable, or lacks food, the model returns a low-confidence warning (UnrecognizedFoodException), prompting the user to re-upload or enter values manually.

F03 – Sleep Architecture & Circadian Recovery Scorer

- Description: Computes a daily Recovery & Readiness Index (0–100) using a rolling 7-day weighted baseline of HRV, resting heart rate, and sleep quality (sleep efficiency, REM percentage, and deep sleep duration).
- User Interaction: Displayed automatically on the home dashboard as a circular readiness gauge with a sleep score breakdown card.
- Input: Latest 7 days of stored BiometricRecord entries (HRV in milliseconds, resting HR in bpm, sleep stage durations in minutes).
- Output: A numeric Recovery Index (0–100) along with readiness tier tags (Prime, Balanced, Fatigued, Depleted).
- AI Involvement: Deterministic.
- Expected Workflow: ReadinessCalculator computes z-scores for HRV and deep sleep against the rolling 7-day average, calculates a weighted composite readiness score, stores the state in AthleteProfile, and notifies dashboard observers to redraw the gauge.
- Error/Alternative Cases: If fewer than 3 days of sleep/HRV data exist, the engine falls back to standard physiological default baselines and displays an informational prompt stating the score is an initial approximation.

F04 – Energy Balance & Caloric Sufficiency Arbiter

- Description: Reconciles total daily active calories burned from workouts and wearable data against logged food intake to evaluate whether caloric and protein intake match daily recovery demands.
- User Interaction: The user opens the "Energy & Diet Balance" screen to view an interactive balance card showing "Burned vs. Consumed" alongside an assessment banner.
- Input: Active calories burned (from Apple Watch + logged gym sets), total basal metabolic rate (BMR), and total logged caloric and protein intake for the day.
- Output: An EnergySufficiencyReport showing net energy deficit/surplus, target protein deficit, and a status tag (Sufficed, Under-Fueled, Hyper-Caloric Surplus).
- AI Involvement: Hybrid.
- Expected Workflow: The system computes the net caloric balance deterministically. An LLM-backed nutrition advisor then assesses the balance in the context of the user's primary goal (e.g., hypertrophy vs. fat loss) and training intensity, generating actionable fueling recommendations.
- Error/Alternative Cases: If the user has logged workouts but zero meals by evening, the feature flags an "Under-Fueled Alert" with guidance on post-workout refuel targets.

F05 – Sleep- and Readiness-Calibrated Workout Generator

- Description: Dynamically creates a customized lifting routine that scales exercise selection, total sets, working rep ranges, and target RPE to match that morning's sleep score and readiness index.
- User Interaction: The user navigates to "Gym Planner", selects their split (e.g., Push, Pull, Legs), and clicks "Generate Adapted Workout".
- Input: Target muscle split, current readiness score, sleep duration, and active training strategy (StrengthStrategy vs. HypertrophyStrategy).
- Output: A structured WorkoutRoutine aggregate containing exercises, set volumes, prescribed load percentages, and recovery notes.
- AI Involvement: Hybrid.
- Expected Workflow: The system calculates target volume limits deterministically based on readiness. The active WorkoutStrategy structures a prompt with the user's constraints and queries the LLM. The LLM returns structured JSON that deserializes into a WorkoutRoutine, displaying the session on screen.
- Error/Alternative Cases: If the LLM client encounters a timeout or network outage, the controller catches ModelInferenceException and falls back to a deterministic offline template matched to the readiness tier.

F06 – Interactive Set & Rep Logger with Command Undo/Redo

- Description: Tracks live gym performance during a session, recording weight lifted, actual repetitions completed, and subjective RPE ratings with full undo and redo capabilities.
- User Interaction: The user selects an exercise card, enters weight and reps into text fields, clicks "Log Set", and can click "Undo" or "Redo" to fix accidental clicks.
- Input: Exercise ID, set index, weight in kilograms/pounds, reps performed, and RPE rating (1–10).
- Output: Visual checkmark on the set table, updated session total volume (tonnage), and auto-trigger of the rest timer.
- AI Involvement: Deterministic.
- Expected Workflow: Clicking "Log Set" instantiates a LogSetCommand passed to SessionCommandInvoker. The invoker executes the command, pushes it onto an undo stack, updates the workout session model, and notifies observers.
- Error/Alternative Cases: If negative or non-numeric inputs are entered, inline field validation prevents command construction and presents an alert without mutating session state.

F07 – Biometric-Adaptive Smart Rest & Recovery Timer

- Description: Runs an intra-set rest countdown that automatically scales its duration based on exercise intensity, logged set RPE, and real-time heart rate recovery data.
- User Interaction: Starts automatically when a set is logged; displays an animated countdown timer in the GUI with "+30s" and "Skip" buttons.
- Input: Exercise movement classification (compound vs. isolation), logged RPE, and current heart rate stream from the wearable.
- Output: Visual countdown timer, dynamic rest length adjustments, and an audio/haptic chime when recovery is sufficient.
- AI Involvement: Deterministic.
- Expected Workflow: Upon set completion, SmartRestEngine evaluates the movement demand (e.g., Heavy Deadlift at RPE 9 requires 180s; Bicep Curl at RPE 7 requires 75s). If live heart rate is connected, it holds the timer until heart rate returns below 60% of max, updating GUI observers on every tick.
- Error/Alternative Cases: If the lifter starts the next set early, the system aborts the countdown, logs the actual rest duration taken, and resets the timer for the next set.

F08 – Proactive Overtraining & CNS Fatigue Sentinel

- Description: Monitors multi-week correlations across sleep debt, suppressed HRV, and accumulated lifting tonnage to identify central nervous system (CNS) overreaching and automatically trigger deload protocols.
- User Interaction: If high fatigue is detected, a prominent "Recovery Warning" modal appears on the dashboard with a "View Fatigue Diagnosis" button.
- Input: 14-day chronological window of sleep metrics, HRV baselines, and accumulated workout tonnage.
- Output: A structured diagnostic analysis of systemic fatigue, overtraining risk score (Low, Elevated, Critical), and an option to generate a 7-day deload week.
- AI Involvement: AI-Based.
- Expected Workflow: The controller detects 3 or more consecutive days of suppressed HRV and sleep debt, packaging the 14-day trends into a prompt for the agent. The agent analyzes patterns across the metrics and produces an explanatory diagnostic report.
- Error/Alternative Cases: If biometric logs are missing for several days, the agent highlights the missing timeline entries and issues conservative guidance.

F09 – Natural Language Holistic Training & Diet Coach

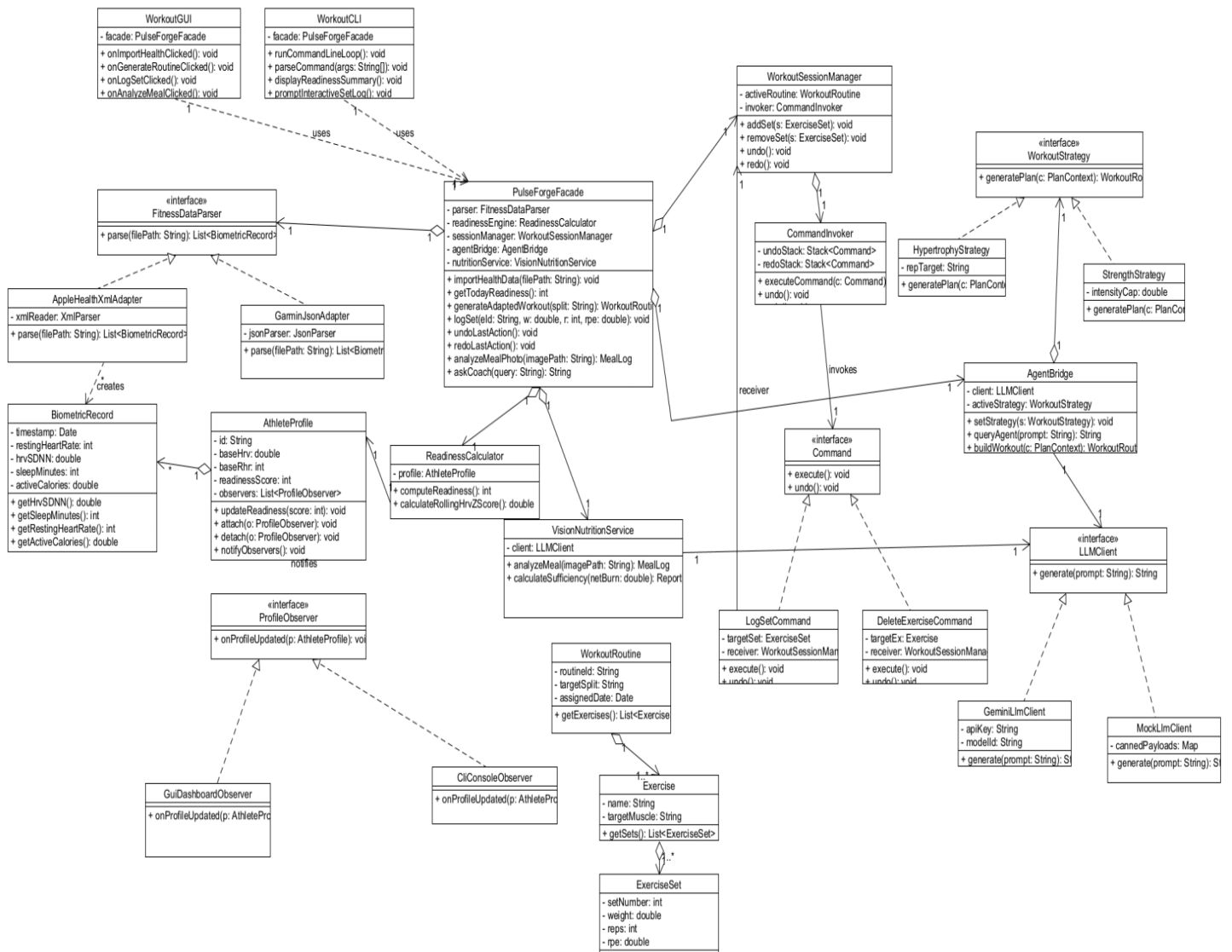
- Description: An interactive conversational coach that answers natural-language queries regarding training modifications, fatigue management, and meal adjustments based on the user's live profile.
- User Interaction: The user types queries into the "AI Coach" chat window (e.g., *"I only slept 5 hours and had a high-calorie lunch—should I still hit heavy squats today?"*) and clicks "Send".
- Input: Natural-language user prompt, paired with current readiness index, sleep duration, today's caloric balance, and recent workout logs.
- Output: Context-aware coaching advice providing workout adaptations and nutrition suggestions grounded in the user's data.
- AI Involvement: AI-Based.
- Expected Workflow: The controller collects active domain state (today's MealLog, BiometricRecord, and WorkoutRoutine), combines it with the user message into an augmented prompt, queries LLMClient, and appends the response to the chat panel.
- Error/Alternative Cases: If the user asks for medical diagnoses or queries unrelated to fitness and recovery, the agent applies guardrails, returns a medical disclaimer, and guides the conversation back to exercise and recovery programming.

F10 – Integrated Fitness, Sleep & Diet Audit Exporter

- Description: Compiles biometric recovery trends, sleep architecture, total training volume, energy balances, and AI coaching insights into an exportable summary document.
- User Interaction: The user clicks "Export Report" on the analytics tab, selects a date range, chooses between Markdown or PDF output, and clicks "Generate".
- Input: Start date, end date, format preference, and target file destination path.
- Output: A formatted report saved to disk containing weekly tables, metric summaries, and coach notes.
- AI Involvement: Deterministic.
- Expected Workflow: ReportGenerator pulls all records across the date range, calculates weekly volume and recovery averages, formats tables, and writes the output file to the specified path.
- Error/Alternative Cases: If file system write access is denied or the directory is invalid, the exporter throws an IOException, keeps data safely in memory, and prompts the user to select an alternate file destination.

Design System using UML

2.1 Class Diagram



1. Adapter Pattern (Structural)

- Design Problem Addressed: The software must ingest biometric records from different wearable platforms. Apple Health exports data as proprietary nested XML (export.xml), whereas other vendors (like Garmin) output JSON or CSV. Directly coupling the core data layer to Apple's XML schema violates the Open-Closed Principle (OCP) and makes it impossible to integrate other wearables without refactoring.
- Participating Classes:
 - FitnessDataParser (<<interface>> / Target)
 - AppleHealthXmlAdapter (Adapter)
 - XmlParser (Adaptee / Java XML parsing library)
 - BiometricRecord (Product / Target Data Format)
- Role Each Class Plays:
 - FitnessDataParser defines the target interface (parse(filePath): List<BiometricRecord>) relied on by application controllers.
 - AppleHealthXmlAdapter implements FitnessDataParser, encapsulates an XmlParser instance, and transforms raw Apple XML tags (such as HKQuantityTypeIdentifierHeartRateVariabilitySDNN) into normalized BiometricRecord objects.
- Why It Is Appropriate: It cleanly decouples vendor-specific data structures from domain logic via object composition. The rest of the codebase works exclusively with uniform BiometricRecord lists regardless of the underlying raw file format.
- What Would Be More Difficult Without It: The import controller would need large conditional chains checking file extensions and manually parsing each schema. Supporting a new smartwatch format would require editing core ingestion logic, creating regression risks in existing parsers.

2. Strategy Pattern (Behavioral)

- Design Problem Addressed: Workout programming algorithms change significantly depending on the athlete's primary training goal (e.g., maximum muscular hypertrophy vs. pure strength). Each approach uses different volume scaling, target rep ranges, and prompt guidelines when communicating with the LLM. Hardcoding these variations into conditional statements inside the workout generator makes the algorithm inflexible and difficult to extend.
- Participating Classes:
 - WorkoutStrategy (<<interface>> / Strategy)
 - HypertrophyStrategy (ConcreteStrategy)
 - StrengthStrategy (ConcreteStrategy)
 - DeloadStrategy (ConcreteStrategy)
 - AgentBridge (Context)
- Role Each Class Plays:

- WorkoutStrategy establishes the contract generatePlan(Context): WorkoutRoutine.
- HypertrophyStrategy implements volume-driven generation (higher total sets, 8–12 rep targets, moderate RPE).
- StrengthStrategy implements intensity-driven generation (lower sets, heavy loads in the 3–5 rep range, higher RPE, longer rest).
- AgentBridge maintains a reference to the active WorkoutStrategy and executes it dynamically at runtime.
- Why It Is Appropriate: It encapsulates each workout generation algorithm into its own class, adhering to the Single Responsibility Principle (SRP) and Open-Closed Principle (OCP). Additional goals (such as endurance or athletic conditioning) can be added without modifying existing generation routines.
- What Would Be More Difficult Without It: The generator class would contain bloated, nested conditional branches checking if (goal == "STRENGTH") or else if (goal == "HYPERTROPHY"). Modifying prompt guidelines or rep thresholds for one goal could inadvertently break others.

3. Facade Pattern (Structural)

- Design Problem Addressed: PulseForge AI contains multiple complex subsystems: file parsing, recovery computation, active workout tracking, vision nutrition analysis, and external LLM communication. Requiring both the graphical interface (WorkoutGUI) and command-line interface (WorkoutCLI) to coordinate these subsystems directly causes tight coupling and duplicate orchestration code across UI boundaries.
- Participating Classes:
 - PulseForgeFacade (Facade)
 - ReadinessCalculator (Subsystem Component)
 - WorkoutSessionManager (Subsystem Component)
 - AgentBridge (Subsystem Component)
 - VisionNutritionService (Subsystem Component)
 - WorkoutGUI / WorkoutCLI (Clients)
- Role Each Class Plays:
 - PulseForgeFacade provides a consolidated, high-level API (importHealthData(), getTodayReadiness(), generateAdaptedWorkout(), logSet(), analyzeMealPhoto(), askCoach()).
 - The subsystem classes carry out the concrete mathematical, stateful, and network operations.
 - Both WorkoutGUI and WorkoutCLI interact exclusively with PulseForgeFacade.
- Why It Is Appropriate: It provides a clear separation of concerns, isolating the presentation tier (GUI and CLI) from the underlying domain and agent services. Internal subsystem modifications will not break the user interfaces.
- What Would Be More Difficult Without It: Every GUI button listener and CLI command loop would need to manually instantiate parsers, call readiness algorithms,

assemble prompts, deserialize JSON payloads, and manage database transactions in sequence. This would duplicate code and make UI maintenance error-prone.

4. Command Pattern (Behavioral)

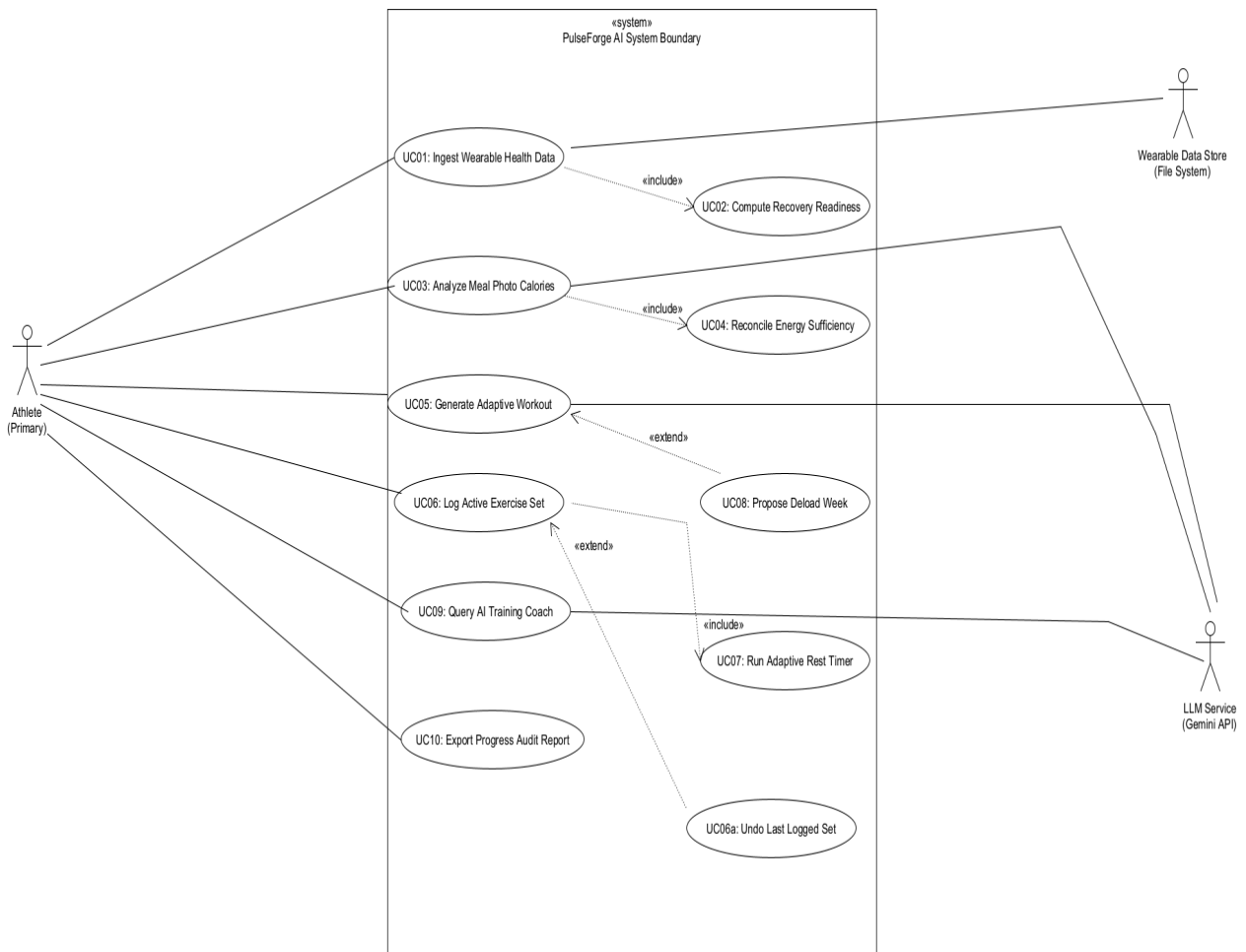
- Design Problem Addressed: In a live workout environment, lifters often mistype weights or reps, log sets prematurely, or remove exercises by mistake. The system must support reliable, multi-level undo and redo operations without tightly coupling UI buttons to the underlying data store.
- Participating Classes:
 - Command (<<interface>> / Command)
 - LogSetCommand (ConcreteCommand)
 - DeleteExerciseCommand (ConcreteCommand)
 - CommandInvoker (Invoker)
 - WorkoutSessionManager (Receiver)
- Role Each Class Plays:
 - Command declares execution and rollback contracts: execute(): void and undo(): void.
 - LogSetCommand captures all data needed to record a set (ExerciseSet details) and knows how to apply or reverse that set on WorkoutSessionManager.
 - CommandInvoker executes incoming commands and manages two stacks (undoStack and redoStack).
 - WorkoutSessionManager acts as the receiver, holding the live active workout session state.
- Why It Is Appropriate: It encapsulates user actions as self-contained objects[cite: 409]. This decouples the invoker (UI action listener) from the receiver (session state) and makes full undo/redo stacks straightforward to implement.
- What Would Be More Difficult Without It: The workout logger would have to overwrite state directly on every button click. Supporting undo would require ad-hoc, error-prone manual rollbacks or saving complete clones of the entire session state after every logged set, wasting memory.

5. Observer Pattern (Behavioral)

- Design Problem Addressed: PulseForge AI is required to support both a Graphical User Interface and a Command-Line Interface. Whenever biometric files are imported, daily readiness scores are recomputed, or active sets are logged, both interfaces—as well as summary dashboard widgets—must update their displays automatically without tightly coupling domain models to specific views.
- Participating Classes:
 - AthleteProfile (ConcreteSubject)
 - ProfileObserver (<<interface>> / Observer)
 - GuiDashboardObserver (ConcreteObserver)
 - CliConsoleObserver (ConcreteObserver)
- Role Each Class Plays:

- AthleteProfile maintains athlete state (HRV baselines, sleep hours, computed Readiness Score) and manages a registered list of ProfileObserver instances. When state updates, it calls its internal notifyObservers() method.
 - ProfileObserver defines the update interface onProfileUpdated(AthleteProfile).
 - GuiDashboardObserver implements the interface to refresh UI gauges, charts, and progress bars.
 - CliConsoleObserver implements the interface to re-render terminal output lines.
- Why It Is Appropriate: It establishes clean one-to-many publish-subscribe communication. The domain layer (AthleteProfile) can notify any number of listening views without holding references to GUI or CLI implementation classes.
- What Would Be More Difficult Without It: AthleteProfile would need explicit references to specific GUI windows and CLI output handlers. Every state modification would require hardcoded calls like guiWindow.refresh() and cliTerminal.print(), causing circular dependencies and preventing either interface from being tested or run independently.

2.2 Use-Case Diagram



UC01: Ingest Wearable Health Data

- Use Case ID: UC01
- Use Case Name: Ingest Wearable Health Data
- Actor(s): Athlete (Primary), Wearable Data Store (Secondary)
- Goal: Import raw biometric files and persist standardized physiological records.
- Preconditions: The wearable device has exported an export.xml or Garmin file accessible on local storage.
- Trigger: Athlete selects "Import Health File" in the GUI or issues pulseforge import <path> in the CLI.
- Main Success Scenario:
 - The athlete selects the file path and confirms the import.
 - The system determines the file format and invokes FitnessDataParser via AppleHealthXmlAdapter.
 - The adapter extracts timestamps, resting HR, HRV (SDNN), sleep stage minutes, and active calories.
 - The system validates record ranges and appends them to AthleteProfile.
 - The system automatically triggers UC02 to recompute baseline readiness.
 - A success confirmation with imported day count displays to the user.
- Alternative/Exception Flows:
 - *3a. Invalid File Format or Corrupt XML:* The adapter catches an XML parse exception and throws InvalidHealthDataException. The system aborts database write, displays an error dialog indicating the corrupted tag or file type, and keeps the previous state unchanged.
- Postconditions: New BiometricRecord entities are persisted to the database and profile baselines are updated.
- Related Feature(s): F01

UC02: Compute Recovery Readiness

- Use Case ID: UC02
- Use Case Name: Compute Recovery Readiness
- Actor(s): Athlete (Primary)
- Goal: Calculate an objective 0–100 Readiness Index from rolling HRV and sleep baselines.
- Preconditions: Athlete profile contains at least one day of logged biometrics.
- Trigger: Invoked automatically following UC01, or manually when the athlete clicks "Recalculate Readiness".
- Main Success Scenario:
 - The calculation engine queries the preceding 7 days of HRV and sleep duration data.
 - The engine computes rolling mean and standard deviation for overnight HRV and sleep duration.

- A weighted z-score composite formula calculates the 0–100 Readiness Index and assigns a category (Prime, Balanced, Fatigued, Depleted).
- AthleteProfile updates its internal score and invokes notifyObservers().
- GuiDashboardObserver and CliConsoleObserver update gauges and text output.
- Alternative/Exception Flows:
 - *1a. Less than 3 days of historical records:* Standard deviation cannot be computed reliably. The system uses default population baselines, marks the score with an "Approximation" badge, and notifies the athlete to import more history.
- Postconditions: The athlete's daily Readiness Index is stored and UI observers display the refreshed state.
- Related Feature(s): F03

UC03: Analyze Meal Photo Calories & Macros

- Use Case ID: UC03
- Use Case Name: Analyze Meal Photo Calories & Macros
- Actor(s): Athlete (Primary), LLM Service (Secondary)
- Goal: Recognize meal items from an image and log macronutrients and calories.
- Preconditions: Device has network connectivity to access the Gemini API; image file exists.
- Trigger: Athlete selects "Upload Meal Image" in the GUI or runs `pulseforge meal --img <path>` in the CLI.
- Main Success Scenario:
 - Athlete uploads a picture of a meal.
 - VisionNutritionService base64-encodes the image into a multimodal prompt.
 - The system queries LLMClient.generate() requesting structured JSON output.
 - The model returns recognized foods, estimated portion sizes, calories, and macronutrient grams.
 - The GUI displays an editable preview table showing parsed values.
 - Athlete reviews, optionally edits numbers, and clicks "Confirm Meal".
 - The system creates a MealLog entry and automatically triggers UC04.
- Alternative/Exception Flows:
 - *3a. API Connection Timeout:* System displays an alert: *"Network unavailable. Would you like to enter meal calories manually?"* and switches to a manual text entry form.
 - *4a. Blurry or Unrecognized Image:* Model returns low-confidence flags; system prompts: *"Could not clearly identify meal items. Please retake photo or enter manually."*
- Postconditions: A MealLog is saved to daily intake records, updating cumulative calories and protein.
- Related Feature(s): F02

UC04: Reconcile Energy Sufficiency

- Use Case ID: UC04
- Use Case Name: Reconcile Energy Sufficiency
- Actor(s): Athlete (Primary), LLM Service (Secondary)
- Goal: Compare daily calories burned against consumed calories and assess recovery fuel status.
- Preconditions: Active daily calories burned have been calculated and at least one meal is recorded.
- Trigger: Automatically invoked following UC03 or gym workout completion.
- Main Success Scenario:
 - System computes net balance:
(*ActiveBurn* + *BMR*) – *TotalConsumedCalories*.
 - System verifies protein intake against the target ratio (e.g., 1.8g/kg).
 - LLMClient analyzes net energy along with today's training intensity and generates fueling feedback.
 - The dashboard displays the status (Sufficed, Under-Fueled, Hyper-Caloric Surplus) alongside recommended meal adjustments.
- Alternative/Exception Flows:
 - 1a. *Workouts logged but 0 meals recorded by 7 PM*: System flags an immediate "Under-Fueled Alert" suggesting post-workout carbohydrate and protein targets.
- Postconditions: Daily EnergySufficiencyReport is saved and displayed on the nutrition dashboard.
- Related Feature(s): F04

UC05: Generate Adaptive Workout Routine

- Use Case ID: UC05
- Use Case Name: Generate Adaptive Workout Routine
- Actor(s): Athlete (Primary), LLM Service (Secondary)
- Goal: Generate a calibrated resistance training session based on daily recovery scores.
- Preconditions: Daily readiness score has been calculated (UC02).
- Trigger: Athlete selects a muscle split (e.g., "Leg Day") and clicks "Generate Adapted Workout".
- Main Success Scenario:
 - Athlete selects target training split and strategy (HypertrophyStrategy or StrengthStrategy).
 - The controller injects readiness index, sleep duration, and equipment constraints into the strategy context.
 - AgentBridge queries LLMClient.generate() requesting a structured JSON workout routine.
 - The model returns exercise selections, set volume, target reps, and RPE caps adjusted to readiness.

- The system instantiates WorkoutRoutine, Exercise, and ExerciseSet domain objects.
 - The generated workout displays on screen, ready for the user to start.
- Alternative/Exception Flows:
 - *2a. Severe Fatigue Detected (Readiness < 40 for 3+ days):* System triggers extension use case UC08: Propose Deload Week.
 - *3a. LLM API Timeout or Malformed Response:* The system catches ModelInferenceException and falls back to a deterministic offline workout template scaled to the user's readiness level.
- Postconditions: A WorkoutRoutine aggregate is staged in WorkoutSessionManager.
- Related Feature(s): F05

UC06: Log Active Exercise Set

- Use Case ID: UC06
- Use Case Name: Log Active Exercise Set
- Actor(s): Athlete (Primary)
- Goal: Record weight, reps, and RPE for a completed gym set with undo capability.
- Preconditions: A workout routine is active.
- Trigger: Athlete inputs values and clicks "Log Set" or executes pulseforge log --weight 100 --reps 8 --rpe 8.
- Main Success Scenario:
 - Athlete enters completed weight, rep count, and RPE rating.
 - System builds a LogSetCommand referencing the target set and passes it to CommandInvoker.
 - CommandInvoker executes the command and pushes it onto undoStack.
 - WorkoutSessionManager marks the set completed and recalculates session tonnage.
 - The system automatically invokes UC07: Run Adaptive Rest Timer.
 - UI displays the updated set row and total volume.
- Alternative/Exception Flows:
 - *1a. Negative or Non-numeric Input:* UI validation catches the error before command creation, highlighting the field and canceling execution.
 - *Athlete requests rollback:* Extension use case UC06a: Undo Last Logged Set is triggered.
- Postconditions: The set is marked completed in session history, and total volume increases.
- Related Feature(s): **F06**

UC06a: Undo Last Logged Set (Extension)

- Use Case ID: UC06a
- Use Case Name: Undo Last Logged Set
- Actor(s): Athlete (Primary)
- Goal: Revert an incorrectly logged set from session state.
- Preconditions: At least one command exists on `CommandInvoker.undoStack`.
- Trigger: Athlete clicks the "Undo" button or types `pulseforge undo` in the CLI.
- Main Success Scenario:
 - Athlete triggers undo.
 - `CommandInvoker.undo()` pops `LogSetCommand` from the undo stack and calls `command.undo()`.
 - `WorkoutSessionManager` clears completed set metrics and subtracts volume.
 - The command is pushed onto `redoStack`.
 - UI unchecks the set row and updates session tonnage.
- Alternative/Exception Flows:
 - *1a. Undo Stack is Empty:* The "Undo" button is disabled; CLI outputs: "No actions to undo."
- Postconditions: Session state is restored to the point immediately prior to the undone set.
- Related Feature(s): **F06**

UC07: Run Adaptive Rest Timer

- Use Case ID: UC07
- Use Case Name: Run Adaptive Rest Timer
- Actor(s): Athlete (Primary)
- Goal: Provide an intra-set countdown timer scaled to exercise demand and RPE.
- Preconditions: Triggered directly by completing a set in UC06.
- Trigger: Execution completion of `LogSetCommand`.
- Main Success Scenario:
 - Timer module evaluates exercise type (compound vs. isolation) and logged RPE.
 - Timer calculates duration (e.g., Squats @ RPE 9 = 180s; Curls @ RPE 7 = 60s).
 - Timer starts and sends second-by-second tick updates to GUI observers.
 - Upon reaching 0, the system plays an audio chime and marks the athlete ready for the next set.
- Alternative/Exception Flows:
 - *Athlete clicks "Skip" or starts next set early:* System cancels remaining countdown time, records actual rest elapsed, and resets timer state.
- Postconditions: Rest interval duration is logged into set metadata.
- Related Feature(s): **F07**

UC08: Propose Deload Week (Extension)

- Use Case ID: UC08
- Use Case Name: Propose Deload Week
- Actor(s): Athlete (Primary), LLM Service (Secondary)
- Goal: Generate a low-fatigue 7-day training schedule to recover from detected CNS overtraining.
- Preconditions: Accumulated fatigue conditions met in UC05 (e.g., Readiness < 40 for 3 consecutive days).
- Trigger: System detects fatigue threshold during UC05 or athlete clicks "Plan Deload Week".
- Main Success Scenario:
 - System displays an overtraining alert advising a deload week.
 - Athlete confirms "Generate Deload Plan".
 - AgentBridge switches to DeloadStrategy.
 - The agent queries LLMClient, reducing total weekly sets by 50% and capping RPE at 6–7.
 - System generates a 7-day schedule with mobility sessions and light technique work.
 - Schedule is saved to the athlete's workout calendar.
- Alternative/Exception Flows:
 - *2a. Athlete Declines Deload:* System logs the override, suppresses the deload warning for 72 hours, and lowers recommended working RPE caps by 1.
- Postconditions: The upcoming 7-day workout calendar is populated with deload sessions.
- Related Feature(s): **F08**

UC09: Query AI Training Coach

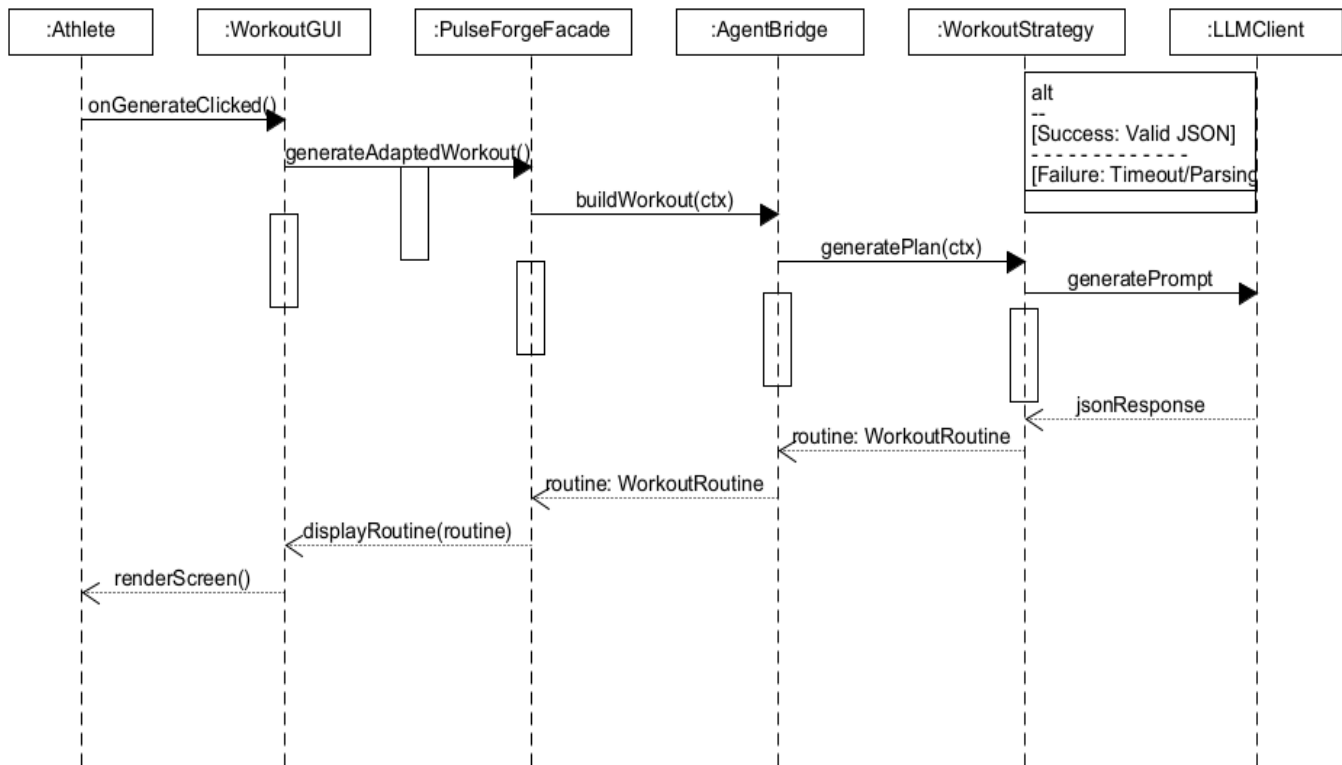
- Use Case ID: UC09
- Use Case Name: Query AI Training Coach
- Actor(s): Athlete (Primary), LLM Service (Secondary)
- Goal: Receive context-aware coaching advice regarding exercise substitutions, form, or fatigue.
- Preconditions: System has access to the user's active session state and biometric profile; network connection is active.
- Trigger: Athlete types a message into the coach chat box and clicks "Send".
- Main Success Scenario:
 - Athlete enters a training question (e.g., *"My lower back is tight—what should I replace Romanian Deadlifts with?"*).
 - PulseForgeFacade gathers current context: daily readiness, active muscle soreness flags, and current workout routine.
 - AgentBridge constructs an augmented prompt and invokes LLMClient.generate().

- LLM analyzes constraints and returns exercise substitutions that target the hamstrings with minimal spinal loading.
 - Coaching response renders in the GUI chat window.
- Alternative/Exception Flows:
 - *1a. Athlete Asks Medical Diagnosis Question:* Agent detects clinical health query, returns a medical disclaimer indicating it cannot diagnose injuries, and advises consulting a healthcare professional.
- Postconditions: The conversation turn is added to chat memory history.
- Related Feature(s): **F09**

UC10: Export Progress Audit Report

- Use Case ID: UC10
- Use Case Name: Export Progress Audit Report
- Actor(s): Athlete (Primary)
- Goal: Generate and save a comprehensive multi-week training, sleep, and diet audit to a local file.
- Preconditions: Stored workout sessions, biometric readings, and meal records exist within the selected date window.
- Trigger: Athlete clicks "Export Audit Report" in the analytics tab.
- Main Success Scenario:
 - Athlete selects a date range (e.g., "Last 30 Days") and format ("Markdown" or "PDF").
 - ReportGenerator retrieves records across the date window.
 - System calculates rolling averages for sleep, HRV, total volume, and net calorie balances.
 - Report is formatted into tables and written to the selected destination path.
 - UI displays a success alert with the saved file path.
- Alternative/Exception Flows:
 - *4a. Destination Directory Unwritable / Disk Full:* System catches IOException, displays an error prompt: *"Unable to write file to selected folder. Please choose a different directory."*, and prompts the file chooser dialog again.
- Postconditions: An audit report document is written to local storage.
- Related Feature(s): **F10**

2.3 Sequence Diagrams



Feature To Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Patterns
F01	Health & Wearable Biometric Ingestion	Deterministic	UC01	WorkoutGUI, PulseForgeFacade, FitnessDataParser, AppleHealthXmlAdapter, GarminJsonAdapter, BiometricRecord	importHealthData(), parse(), readXML()	SD01	Adapter, Facade
F02	Multimodal Meal Photo Calorie & Macro Analyzer	AI-Based	UC03	WorkoutGUI, PulseForgeFacade, VisionNutritionService, LLMClient, GeminiLlmClient, MealLog	analyzeMealPhoto(), analyzeMeal(), generate()	SD02(Variant)	Facade
F03	Sleep & Circadian Recovery Scorer	Deterministic	UC02	PulseForgeFacade, ReadinessCalculator, AthleteProfile, ProfileObserver, GuiDashboardObserver, CliConsoleObserver	computeReadiness(), calculateRollingHrvZScore(), updateReadiness(), notifyObservers(), onProfileUpdated()	SD01	Observer, Facade
F04	Energy Balance & Caloric Sufficiency Arbiter	Hybrid	UC04	WorkoutGUI, PulseForgeFacade, VisionNutritionService, LLMClient, AthleteProfile	calculateSufficiency(), queryAgent(), generate()	SD02(Variant)	Facade
F05	Sleep & Readiness Calibrated Workout Generator	Hybrid	UC05	WorkoutGUI, PulseForgeFacade, AgentBridge, WorkoutStrategy, HypertrophyStrategy, StrengthStrategy, LLMClient, GeminiLlmClient, WorkoutRoutine	generateAdaptedWorkout(), buildWorkout(), generatePlan(), generate(), displayRoutine()	SD02	Strategy, Facade
F06	Interactive Set & Rep Logger with Undo/Redo	Deterministic	UC06, UC06A	WorkoutGUI, PulseForgeFacade, WorkoutSessionManager, CommandInvoker, Command, LogSetCommand	logSet(), executeCommand(), execute(), undoLastAction(), undo()	SD03	Command, Facade
F07	Biometric-Adaptive Smart Rest Timer	Deterministic	UC07	WorkoutGUI, WorkoutSessionManager, LogSetCommand, RestTimer	execute(), startTimer(), start(), tick()	SD03	Command, Observer

F08	Proactive Deload Week Planner	Hybrid	UC08	PulseForgeFacade, AgentBridge, WorkoutStrategy, DeloadStrategy, LLMClient, WorkoutRoutine	setStrategy(), generatePlan(), generate(), buildWorkout()	SD02(Va raint)	Strategy, Facade
F09	Natural Language Training & Diet Coach	AI-Based	UC09	WorkoutGUI, PulseForgeFacade, AgentBridge, LLMClient, GeminiLlmClient, AthleteProfile	askCoach(), queryAgent(), generate()	SD02(Va raint)	Facade
F10	Integrated Fitness, Sleep & Diet Audit Exporter	Deterministic	UC10	WorkoutGUI, PulseForgeFacade, ReportGenerator, AthleteProfile, WorkoutSessionManager	exportReport(), compileAudit(), writeToFile()	SD01(Va raint)	Facade

Feature Realization

F01 Health & Wearable Biometric Ingestion

- Related Use Case: UC01 — Ingest Wearable Health Data
- Related Sequence Diagram: SD01 — Ingest Biometric Data & Compute Readiness
- Classes involved:
 - WorkoutGUI — boundary window providing the file chooser and import actions.
 - PulseForgeFacade — high-level system interface coordinating ingestion across subsystems.
 - FitnessDataParser — common interface defining the file parsing contract.
 - AppleHealthXmlAdapter — concrete adapter parsing Apple Health XML nodes.
 - GarminJsonAdapter — alternative adapter converting wearable JSON exports.
 - BiometricRecord — domain entity holding normalized resting HR, HRV, sleep, and active burn data.
 - AthleteProfile — aggregate root that stores historical biometric entries.
- Important methods:
 - WorkoutGUI.onImportHealthClicked()
 - PulseForgeFacade.importHealthData()
 - FitnessDataParser.parse()
 - AppleHealthXmlAdapter.parse()
 - AthleteProfile.storeRecords()
- Execution: When the user selects "Import Health File" in the GUI, onImportHealthClicked() forwards the selected path to PulseForgeFacade.importHealthData(). The facade delegates the file to FitnessDataParser via AppleHealthXmlAdapter.parse(). The adapter extracts raw sensor tags, constructs normalized BiometricRecord domain instances, and returns them to the facade. The facade persists these records into AthleteProfile and invokes recovery recalculation.

F02 Multimodal Meal Photo Calorie & Macro Analyzer

- Related Use Case: UC03 — Analyze Meal Photo Calories
- Related Sequence Diagram: SD02 (Variant) — Multimodal Vision Analysis
- Classes involved:
 - WorkoutGUI — presents the meal image upload dialog and preview tables.
 - PulseForgeFacade — entry point for dietary requests.
 - VisionNutritionService — service packaging images and deserializing nutrition payloads.
 - LLMClient — interface defining text and multimodal inference contracts.
 - GeminiLlmClient — concrete client transmitting requests to the Gemini 1.5 Flash API.

- MealLog — domain model storing parsed foods, calories, and macronutrient grams.
- Important methods:
 - WorkoutGUI.onAnalyzeMealClicked()
 - PulseForgeFacade.analyzeMealPhoto()
 - VisionNutritionService.analyzeMeal()
 - LLMClient.generate()
 - MealLog.create()
- Execution: When the user uploads a photo of a meal and clicks "Analyze Meal", WorkoutGUI triggers PulseForgeFacade.analyzeMealPhoto(). The facade invokes VisionNutritionService.analyzeMeal(), which base64-encodes the image and formats a multimodal prompt. The prompt is sent to LLMClient.generate() using GeminiLlmClient. The model infers the food items, portions, calories, and macronutrients, returning structured JSON. The service deserializes this into a MealLog object and returns it to WorkoutGUI for user verification and confirmation.

F03 Sleep Architecture & Circadian Recovery Scorer

- Related Use Case: UC02 — Compute Recovery Readiness
- Related Sequence Diagram: SD01 — Ingest Biometric Data & Compute Readiness
- Classes involved:
 - PulseForgeFacade — coordinates calculation triggers.
 - ReadinessCalculator — mathematical engine computing rolling baselines and z-scores.
 - AthleteProfile — concrete subject holding profile state and recovery scores.
 - ProfileObserver — observer interface defining update notification hooks.
 - GuiDashboardObserver — concrete observer updating UI dials and gauges.
 - CliConsoleObserver — concrete observer updating terminal metrics.
- Important methods:
 - PulseForgeFacade.getTodayReadiness()
 - ReadinessCalculator.computeReadiness()
 - AthleteProfile.updateReadiness()
 - AthleteProfile.notifyObservers()
 - ProfileObserver.onProfileUpdated()
- Execution:

Following health file ingestion or upon manual recalculation, PulseForgeFacade.getTodayReadiness() calls ReadinessCalculator.computeReadiness(). The calculator evaluates the past 7 days of HRV and sleep duration against the user's historical rolling mean and standard deviation. It calculates a composite 0–100 Readiness Score and updates the domain state via AthleteProfile.updateReadiness(). The profile executes notifyObservers(), which calls onProfileUpdated() on GuiDashboardObserver and CliConsoleObserver to refresh the displays automatically.

F04 Energy Balance & Caloric Sufficiency Arbiter

- Related Use Case: UC04 — Reconcile Energy Sufficiency
- Related Sequence Diagram: SD02 (Variant) — Hybrid Decision Engine
- Classes involved:
 - WorkoutGUI — renders caloric balance charts and recommendation banners.
 - PulseForgeFacade — provides access to daily energy reconciliation.
 - VisionNutritionService — determines net energy surplus or deficit.
 - AthleteProfile — supplies BMR and historical active energy burned.
 - LLMClient — generates context-aware dietary feedback.
 - Report — domain entity encapsulating net balances, protein targets, and advisory status.
- Important methods:
 - WorkoutGUI.displayEnergyReport()
 - PulseForgeFacade.reconcileEnergyBalance()
 - VisionNutritionService.calculateSufficiency()
 - LLMClient.generate()
- Execution: When a meal is confirmed or workout is finalized, PulseForgeFacade invokes VisionNutritionService.calculateSufficiency(). The service deterministically computes net caloric balance: $(ActiveBurn + BMR) - TotalConsumedCalories$. If an acute deficit or excess exists relative to today's workout tonnage, the service queries LLMClient.generate() with athlete context. The model generates actionable fueling recommendations, which are compiled into a Report and displayed on WorkoutGUI.

F05 Sleep and Readiness-Calibrated Workout Generator

- Related Use Case: UC05 — Generate Adaptive Workout Routine
- Related Sequence Diagram: SD02 — Adaptive Workout Generation Sequence Diagram
- Classes involved:
 - WorkoutGUI — collects muscle split selection and user preferences.
 - PulseForgeFacade — coordinates workout planning requests.
 - AgentBridge — agent controller managing prompt assembly, strategies, and client calls.
 - WorkoutStrategy — strategy interface declaring the plan generation contract.
 - HypertrophyStrategy / StrengthStrategy — concrete algorithms enforcing rep ranges and load targets.
 - LLMClient — executes inference calls against Gemini.
 - WorkoutRoutine — domain aggregate containing ordered exercises and target sets.
- Important methods:
 - WorkoutGUI.onGenerateRoutineClicked()
 - PulseForgeFacade.generateAdaptedWorkout()

- AgentBridge.buildWorkout()
 - WorkoutStrategy.generatePlan()
 - LLMClient.generate()
- Execution: When the user selects a target split and clicks "Generate Adapted Workout", WorkoutGUI calls PulseForgeFacade.generateAdaptedWorkout(). The facade forwards parameters to AgentBridge.buildWorkout(), which delegates to the active WorkoutStrategy.generatePlan() (e.g., HypertrophyStrategy). The strategy injects the user's readiness score, sleep duration, and target muscle groups into a structured prompt and sends it to LLMClient.generate(). The returned JSON routine is deserialized into a WorkoutRoutine aggregate and displayed on the interface. If the API times out, AgentBridge catches ModelInferenceException and loads a deterministic fallback routine.

F06 Interactive Set & Rep Logger with Command Undo/Redo

- Related Use Case: UC06 — Log Active Exercise Set, UC06a — Undo Last Logged Set
- Related Sequence Diagram: SD03 — Live Session Command Execution & Undo
- Classes involved:
 - WorkoutGUI — provides input fields for weight, reps, and RPE, alongside Undo/Redo buttons.
 - PulseForgeFacade — receives set entries and routes undo/redo requests.
 - WorkoutSessionManager — receiver holding the live workout session and tonnage state.
 - CommandInvoker — invoker tracking the undoStack and redoStack.
 - Command — command interface declaring execute() and undo().
 - LogSetCommand — concrete command encapsulating set data and its inverse operation.
- Important methods:
 - WorkoutGUI.onLogSetClicked()
 - PulseForgeFacade.logSet()
 - CommandInvoker.executeCommand()
 - LogSetCommand.execute()
 - PulseForgeFacade.undoLastAction()
 - CommandInvoker.undo()
 - LogSetCommand.undo()
- Execution: When the user inputs weight, reps, and RPE and clicks "Log Set", WorkoutGUI calls PulseForgeFacade.logSet(). The facade creates a LogSetCommand and passes it to CommandInvoker.executeCommand(). The invoker executes the command, pushing it onto undoStack. The command calls WorkoutSessionManager.addSet(), updating total tonnage and checking off the completed set in the UI. When the user clicks "Undo", PulseForgeFacade.undoLastAction() calls CommandInvoker.undo(), which pops LogSetCommand and invokes its undo() method to revert the session state cleanly.

F07 Biometric-Adaptive Smart Rest & Recovery Timer

- Related Use Case: UC07 — Run Adaptive Rest Timer
- Related Sequence Diagram: SD03 — Live Session Command Execution & Undo
- Classes involved:
 - WorkoutGUI — displays the visual circular countdown timer and interval controls.
 - WorkoutSessionManager — triggers the recovery interval after a set is committed.
 - LogSetCommand — concrete command that invokes timer initiation upon execution.
 - RestTimer — active thread/clock calculating recovery duration and emitting tick pulses.
- Important methods:
 - LogSetCommand.execute()
 - WorkoutSessionManager.startTimer()
 - RestTimer.start()
 - RestTimer.tick()
 - WorkoutGUI.updateTimerDisplay()
- Execution: Immediately after LogSetCommand.execute() mutates the workout session, it invokes WorkoutSessionManager.startTimer(). The manager passes the completed set's RPE and exercise movement classification (compound vs. isolation) to RestTimer. The timer calculates the rest interval dynamically, begins a countdown loop, and notifies GUI observers on each clock tick until 0 is reached, sounding an alert to prompt the lifter for their next set.

F08 Proactive Deload Week Planner

- Related Use Case: UC08 — Propose Deload Week
- Related Sequence Diagram: SD02 (Variant) — Strategy-Based Deload Synthesis
- Classes involved:
 - WorkoutGUI — presents deload recommendations and confirmation dialogues.
 - PulseForgeFacade — coordinates periodization plan requests.
 - AgentBridge — switches active strategies dynamically at runtime.
 - WorkoutStrategy — strategy interface.
 - DeloadStrategy — concrete strategy implementing low-intensity, low-volume periodization rules.

- LLMClient — queries the LLM to structure mobility work and modified lifting loads.
 - WorkoutRoutine — models the 7-day deload schedule.
- Important methods:
 - AgentBridge.setStrategy()
 - PulseForgeFacade.planDeloadWeek()
 - DeloadStrategy.generatePlan()
 - LLMClient.generate()
 - WorkoutSessionManager.stageRoutine()
- Execution: When systemic fatigue indicators trigger an overtraining alert or the user clicks "Plan Deload Week", PulseForgeFacade instructs AgentBridge to swap its current strategy reference using setStrategy(new DeloadStrategy()). The facade then calls AgentBridge.buildWorkout(), triggering DeloadStrategy.generatePlan(). The strategy prompts LLMClient.generate() to reduce weekly training volume by 50% and cap intensity at $RPE \leq 7$. The resulting schedule is deserialized into a 7-day WorkoutRoutine calendar.

F09 Natural Language Holistic Training & Diet Coach

- Related Use Case: UC09 — Query AI Training Coach
- Related Sequence Diagram: SD02 (Variant) — Conversational Agent Flow
- Classes involved:
 - WorkoutGUI — chat dialog interface capturing user text and rendering coach replies.
 - PulseForgeFacade — routes coach queries to the backend agent.
 - AgentBridge — aggregates contextual domain state and constructs prompt messages.
 - AthleteProfile — supplies current readiness, fatigue, and recovery context.
 - WorkoutSessionManager — provides current exercises and logged set data.
 - LLMClient — executes conversational generation requests.
- Important methods:
 - WorkoutGUI.onAskCoachClicked()
 - PulseForgeFacade.askCoach()
 - AgentBridge.queryAgent()
 - LLMClient.generate()
- Execution: The user enters an open-ended training question (e.g., asking for an exercise substitution) into the GUI chat panel and clicks "Send". WorkoutGUI calls PulseForgeFacade.askCoach(query), which delegates to AgentBridge.queryAgent(). The bridge queries AthleteProfile and WorkoutSessionManager to gather live athlete metrics (today's readiness, active sore muscles, exercises already completed). It merges these facts with the user prompt and queries LLMClient.generate(). The response text is returned through the facade and appended to the chat interface.

F10 Integrated Fitness, Sleep & Diet Audit Exporter

- Related Use Case: UC10 — Export Progress Audit Report
- Related Sequence Diagram: SD01 (Variant) — Audit Data Compilation & File I/O
- Classes involved:
 - WorkoutGUI — collects date range, format selection (Markdown/PDF), and destination path.
 - PulseForgeFacade — triggers report compilation.
 - ReportGenerator — aggregates multi-domain domain objects and formats structured output tables.
 - AthleteProfile — provides historical sleep, HRV, and readiness metrics across the date range.
 - WorkoutSessionManager — supplies historical tonnage and exercise progression records.
- Important methods:
 - WorkoutGUI.onExportReportClicked()
 - PulseForgeFacade.exportReport()
 - ReportGenerator.compileAudit()
 - ReportGenerator.writeToFile()
- Execution: When the user selects a date range, picks a format, and clicks "Export Report", WorkoutGUI calls PulseForgeFacade.exportReport(). The facade instructs ReportGenerator.compileAudit() to pull historical biometric records from AthleteProfile and workout sessions from WorkoutSessionManager. The generator calculates rolling averages for sleep, HRV, and net energy balance, formats the data into structured tables, and executes writeToFile() to output the document to disk. A confirmation alert displays on the interface